



UNIVERSITAS MULTI DATA PALEMBANG

Fakultas Ilmu Komputer dan Rekayasa

Program Studi Sistem Informasi

DOKUMENTASI BUKTI PEKERJAAN

Kuis Refactor: Arsitektur Backend Terstruktur

Route, Controller, Model, dan Middleware pada RESTful API Tugas 1

Pengembangan Aplikasi Web II | SUB-CPMK-6 | Kelas SI5C

Nama	Rizqi Fajri Sya'Ban
NPM	2428240109
Kelas	SI5C
Topik Tugas 1	Restoran
Entitas utama	Menu
Tautan repositori	https://github.com/rizqifajri25/si5c_kuis1_RizqiFajriSya-ban.git
Dosen pengampu	Nur Rachmat, M.Kom.
Tanggal pengumpulan	6 Oktober 2026

Petunjuk Pengisian

Dokumen ini adalah bukti bahwa Anda sendiri yang menyelesaikan *refactor* Tugas 1 ke arsitektur backend terstruktur. Dosen menilai dari dua hal: tangkapan layar (*screenshot*) yang membuktikan program berjalan, dan penjelasan Anda yang membuktikan bahwa Anda memahami kode tersebut.

A. Cara mengisi

1. Ganti seluruh tulisan dalam tanda kurung siku, misalnya [sumber-daya] dan [Entitas], dengan topik Tugas 1 Anda. Contoh: untuk topik Buku, [sumber-daya] menjadi buku.
2. Setiap langkah memiliki tiga bagian: *screenshot*, keterangan gambar, dan kotak **Penjelasan**. Ketiganya wajib diisi.
3. Tulis penjelasan dengan kalimat Anda sendiri. Penjelasan yang hanya menyalin nama berkas atau tidak menyebut kode topik Anda dianggap belum mengerti.
4. Pada pengujian, gunakan data milik topik Anda dan masukkan identitas Anda (nama atau NIM) pada salah satu field teks pada permintaan POST. Ini menjadi tanda bahwa gambar berasal dari pekerjaan Anda.
5. Simpan hasil akhir sebagai PDF dengan nama PAW2_Kuis2_SI5C_[NIM]_[Nama].pdf, lalu kumpulkan melalui LMS sesuai batas waktu yang ditentukan dosen.

B. Syarat tangkapan layar

Jenis	Yang harus terlihat
Terminal	Perintah yang diketik, seluruh keluarannya, dan nama folder project pada baris perintah.
Kode	Tidak perlu ditangkap layar. Kode dinilai dari tautan repositori GitHub.
Pengujian <i>endpoint</i>	Metode (GET, POST, PUT, DELETE), alamat lengkap, header yang dikirim, body (bila ada), kode status, dan isi respons. Boleh memakai Postman, Thunder Client, REST Client, atau curl di terminal.
Larangan	Gambar dipotong sehingga status atau isi respons tidak terlihat, gambar buram, gambar hasil unduhan dari teman atau internet, dan gambar yang diedit.

Cara mengambil screenshot: Windows tekan Win + Shift + S; macOS tekan Cmd + Shift + 4; Linux gunakan PrtSc atau aplikasi tangkapan layar bawaan. Tempel gambar ke kotak pada dokumen ini, lalu atur lebarnya agar seluruh tulisan masih terbaca.

C. Susunan dokumen

Bab	Isi	Bukti utama
Bab 1	Instalasi	1 screenshot terminal
Bab 2	Struktur folder dan ringkasan kode	1 screenshot struktur folder, tabel penjelasan
Bab 3	Menjalankan server	1 screenshot terminal

Bab	Isi	Bukti utama
Bab 4	Pengujian endpoint (5 berhasil, 4 gagal)	9 screenshot
Bab 5	Rekapitulasi hasil uji	Tabel rekap
Bab 6	Repositori kode	Tautan GitHub
Bab 7	Refleksi	Dua jawaban tertulis

Seluruh dokumen hanya memerlukan **12 tangkapan layar**. Ambil tangkapan layar sambil mengerjakan, bukan di akhir, dan sisakan sekitar 25 menit dari waktu kuis untuk melengkapi penjelasan.

Identitas Topik dan Rancangan Endpoint

Bagian ini menjembatani topik Tugas 1 Anda dengan contoh yang dipakai pada soal. Isilah sebelum mengerjakan bab berikutnya. Seluruh tanda [...] pada bab selanjutnya mengacu ke tabel ini.

Data topik

Topik Tugas 1	Restoran
Entitas utama (tunggal)	Menu
Alamat dasar	/menu-items
Field data	<pre>let menu = [{ id: 1, nama: 'Es Teh Manis', kategori: 'Minuman', harga: "5000", tersedia: "true", pedas: "false" }, { id: 2, nama: 'Brown Sugar Boba', kategori: 'Minuman', harga: "10000", tersedia: "true", pedas: "false" }, { id: 3, nama: 'Croissant', kategori: 'Makanan', harga: "15000", tersedia: "false", pedas: "false" }, { id: 4, nama: 'Kentang Goreng', kategori: 'Makanan', harga: "10000", tersedia: "true", pedas: "false" }, { id: 5, nama: 'Mie Ayam Pedas', kategori: 'Makanan', harga: "15000", tersedia: "true", pedas: "true" }];</pre>
Field wajib saat POST	
Query string untuk filter	
Data awal	
Modul lain (jika ada)	

Daftar endpoint

Tuliskan seluruh *endpoint* pada API Anda. Kolom "API key" diisi Ya bila rute dilindungi cekApiKey.

No	Metode	Alamat	API key	Status sukses	Fungsi
1	GET	/[sumber-daya]	Tidak	200	Ambil semua data (dengan filter bila ada)
2	GET	/[sumber-daya]/:id	Tidak	200	Ambil satu data
3	POST	/[sumber-daya]	Ya	201	Tambah data
4	PUT	/[sumber-daya]/:id	Ya	200	Ubah data
5	DELETE	/[sumber-daya]/:id	Ya	204	Hapus data

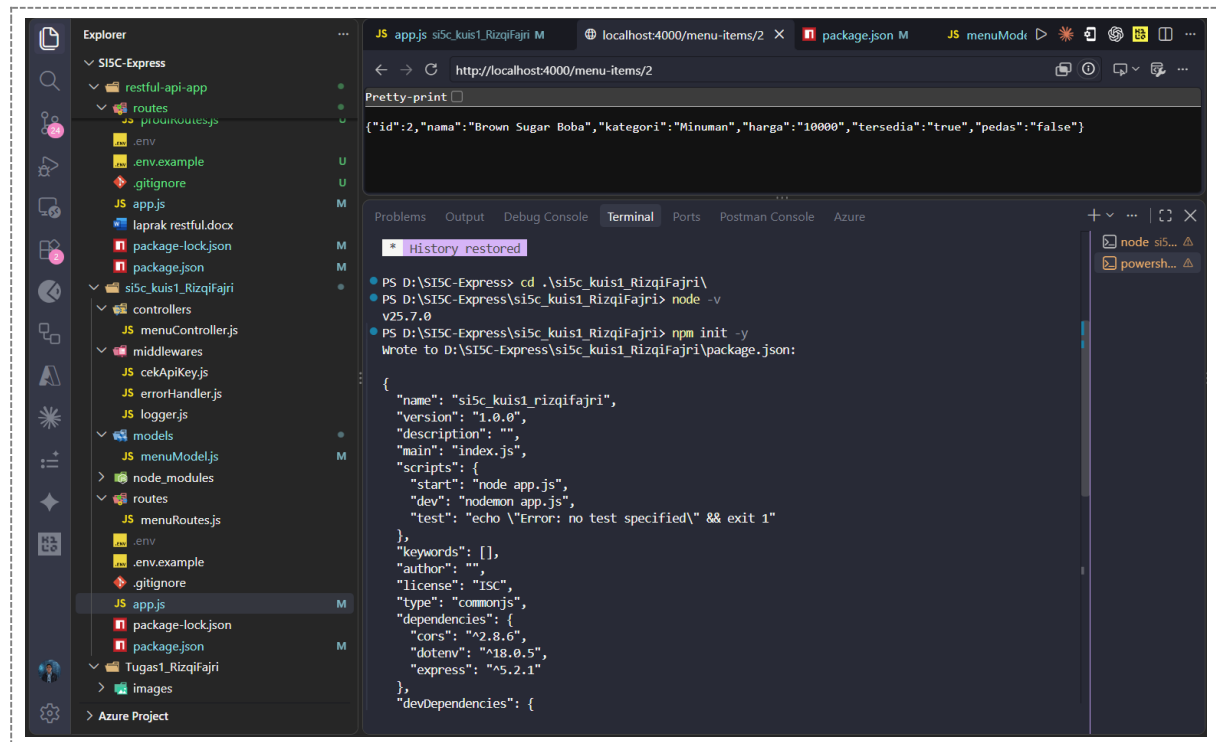
Jika topik Anda memiliki *endpoint* tambahan atau modul lain (misalnya relasi antartabel), tambahkan barisnya di sini dan uji pada bagian "Skenario Tambahan" di Bab 4.

Bab 1 Instalasi

Bab ini membuktikan bahwa lingkungan kerja siap dan *project* dibuat dari awal. Cukup **satu** tangkapan layar terminal yang memuat seluruh perintah instalasi.

1.1 Instalasi project

Harus terlihat pada screenshot: perintah `node -v`, `npm -v`, `npm init -y`, dan `npm install express cors dotenv` beserta keluarannya sampai selesai.



Gambar 1. Versi Node.js dan instalasi paket

Penjelasan:

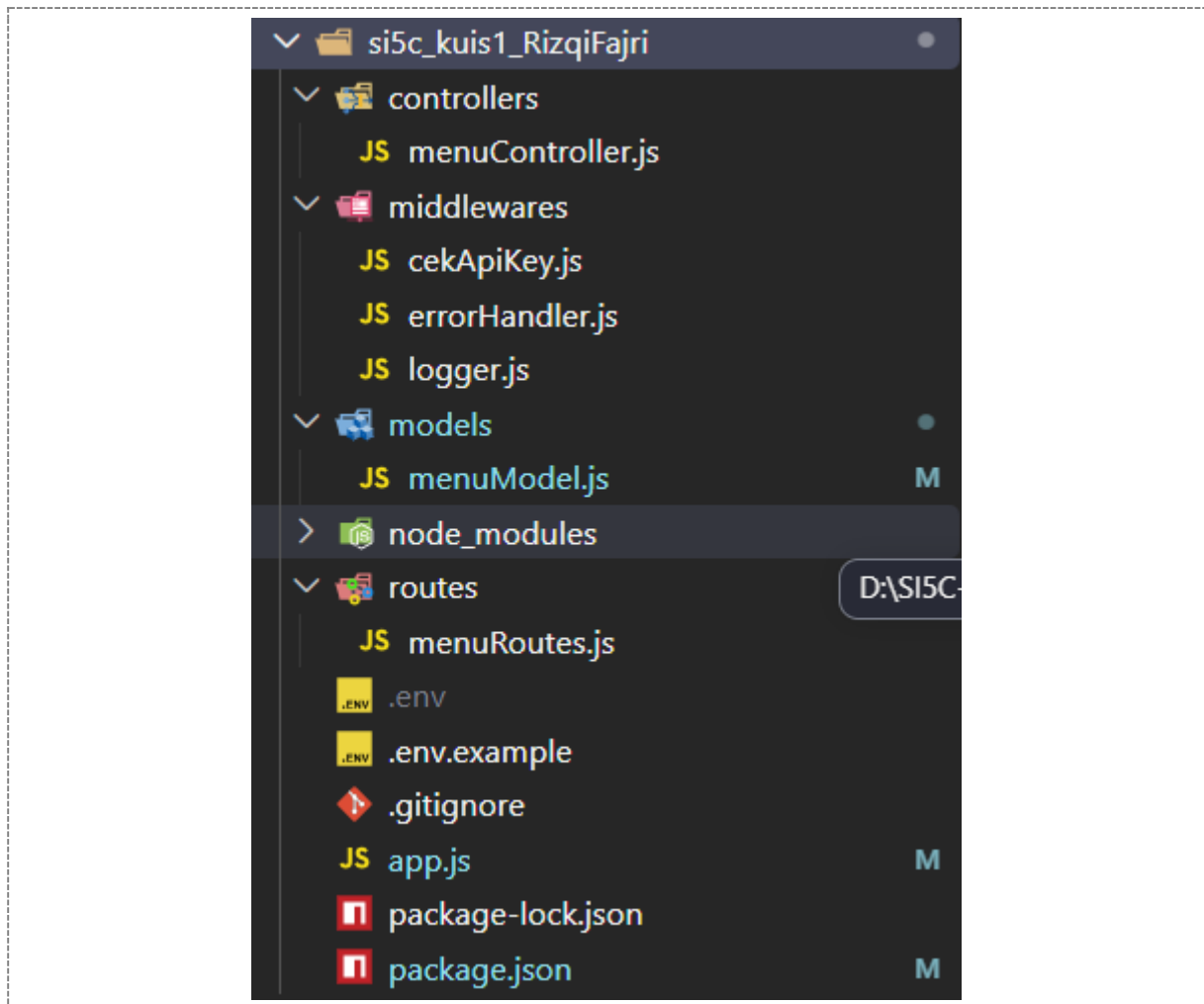
Express Membantu dalam membuat server dan alamat API, misalnya dalam kasus saya ini /menu-items, lalu dotenv berguna untuk membaca pengaturan dari file .env seperti port atau API Key, lalu untuk express 5 itu membutuhkan node.js versi 18 agar fitur dan fungsi express dapat berjalan dengan baik.

Bab 2 Struktur Folder dan Ringkasan Kode

Kode tidak perlu ditangkap layar. Dosen membaca kode langsung dari repositori (lihat Bab 6). Yang diminta di sini adalah **satu** tangkapan layar struktur folder dan penjelasan singkat tiap berkas dengan kalimat Anda sendiri.

2.1 Struktur folder

Harus terlihat pada screenshot: folder routes, controllers, models, dan middlewares beserta isinya. Gunakan panel *Explorer* editor atau perintah tree.



Gambar 2. Struktur folder project

Penjelasan:

Folder Controllers mengatur respon server, lalu folder Middlewares menjalankan pemeriksaan tambahan seperti misalnya error handling, lalu folder Models untuk mengelola data yang ada, lalu folder Routes untuk menentukan alamat API, lalu folder node_modules untuk menyimpan paket yang digunakan.

2.2 Ringkasan tugas setiap berkas

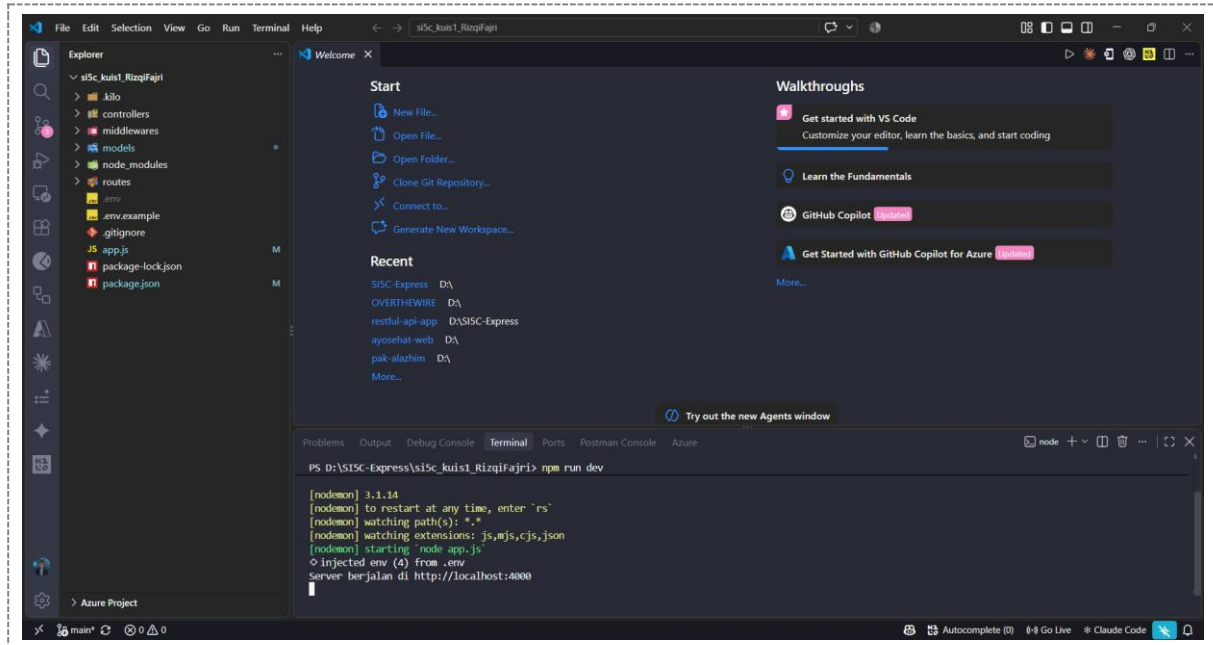
Tulis satu atau dua kalimat per berkas. Sebut nama fungsi atau variabel dari kode Anda.

Berkas	Penjelasan Anda
models/menuModel.js	Variabel menu menyimpan data menu, sedangkan fungsi getAll, getById, create, update, dan remove digunakan untuk melihat, mencari, menambah, mengubah, dan menghapusnya.
Controllers/menuController.js	Fungsi getAll, getById, create, update, dan remove menangani permintaan API dan menggunakan menuModel untuk mengelola data menu.
routes/menuRoutes.js	Variabel router menghubungkan alamat seperti GET / dan POST / ke fungsi di menuController, lalu cekApiKey melindungi operasi tambah, ubah, dan hapus.
middlewares/logger.js	Fungsi logger mencatat waktu, metode, dan alamat setiap permintaan menggunakan console.log.
middlewares/cekApiKey.js	Fungsi cekApiKey memeriksa kunci x-api-key dan menolak permintaan jika kuncinya tidak cocok dengan API KEY yang terdaftar pada .env
middlewares/errorHandler.js	Fungsi errorHttp membuat error dengan kode status, notFoundHandler menangani alamat yang tidak ditemukan, dan errorHandler mengirim respons untuk error.
app.js	Variabel app mengatur server Express dan middleware, sedangkan PORT menentukan port server; menuRoutes dipasang pada alamat /menu-items.

Bab 3 Menjalankan Server

3.1 Menjalankan server

Harus terlihat pada screenshot: perintah `npm start` atau `npm run dev` dan pesan bahwa server berjalan beserta nomor port. Biarkan terminal ini tetap terbuka selama pengujian.



Gambar 3. Server berhasil dijalankan

Penjelasan:

Port diambil dari PORT di `.env`; jika tidak diatur, aplikasi memakai port 3000. Kalau port tersebut sudah dipakai aplikasi lain, server gagal dijalankan dengan error `EADDRINUSE` ("address already in use"); karena kode ini tidak memasang penanganan error khusus untuk `app.listen`.

Bab 4 Pengujian Setiap Endpoint

Pengujian dibagi dua: lima skenario berhasil (T01 sampai T05) dan empat skenario gagal (N01 sampai N04). Skenario berhasil dilakukan **berurutan** karena T04 dan T05 memakai data yang dibuat pada T03. Setiap skenario cukup **satu** tangkapan layar yang memuat permintaan dan respons sekaligus; semua tulisan harus terbaca.

Catatan

Bunyi pesan boleh berbeda dengan contoh sesuai kode Anda, tetapi respons harus berupa JSON dan kode status harus tepat. Jika pesan Anda berbeda, tulis pesan yang sebenarnya pada kolom "Status hasil uji".

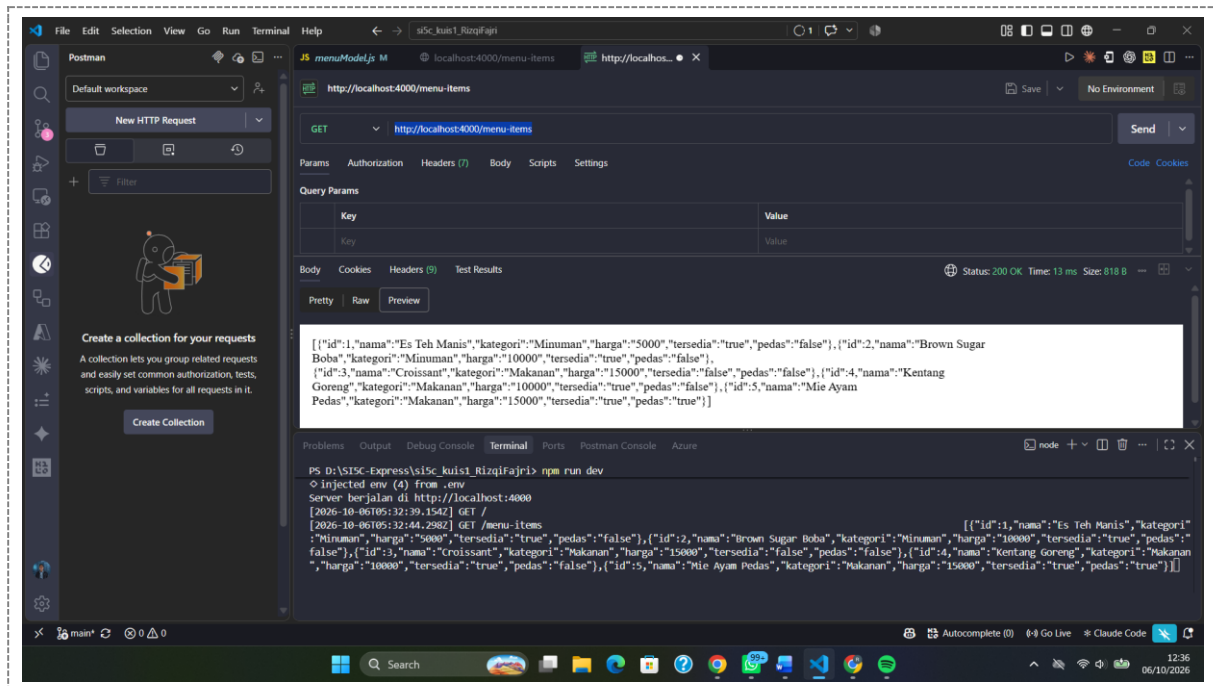
Gunakan ID 99 (atau ID lain yang pasti tidak ada) untuk skenario "tidak ditemukan".

Bagian A. Skenario berhasil

T01 Menampilkan semua data (GET /[sumber-daya])

Harus terlihat pada screenshot: metode GET, alamat lengkap, kode status, dan isi respons berupa larik JSON.

Permintaan (request)	GET /menu-items
Header	(tidak ada)
Body	(tidak ada)
Hasil yang diharapkan	200 OK; isi berupa larik JSON seluruh data.
Status hasil uji	<pre>[{"id":1,"nama":"Es Teh Manis","kategori":"Minuman","harga":"5000","tersedia":"true","pedas":"false"}, {"id":2,"nama":"Brown Sugar Boba","kategori":"Minuman","harga":"10000","tersedia":"true","pedas":"false"}, {"id":3,"nama":"Croissant","kategori":"Makanan","harga":"15000","tersedia":"false","pedas":"false"}, {"id":4,"nama":"Kentang Goreng","kategori":"Makanan","harga":"10000","tersedia":"true","pedas":"false"}, {"id":5,"nama":"Mie Ayam Pedas","kategori":"Makanan","harga":"15000","tersedia":"true","pedas":"true"}]</pre>
Lulus?	Ya



Gambar 4. Hasil GET semua data

Penjelasan:

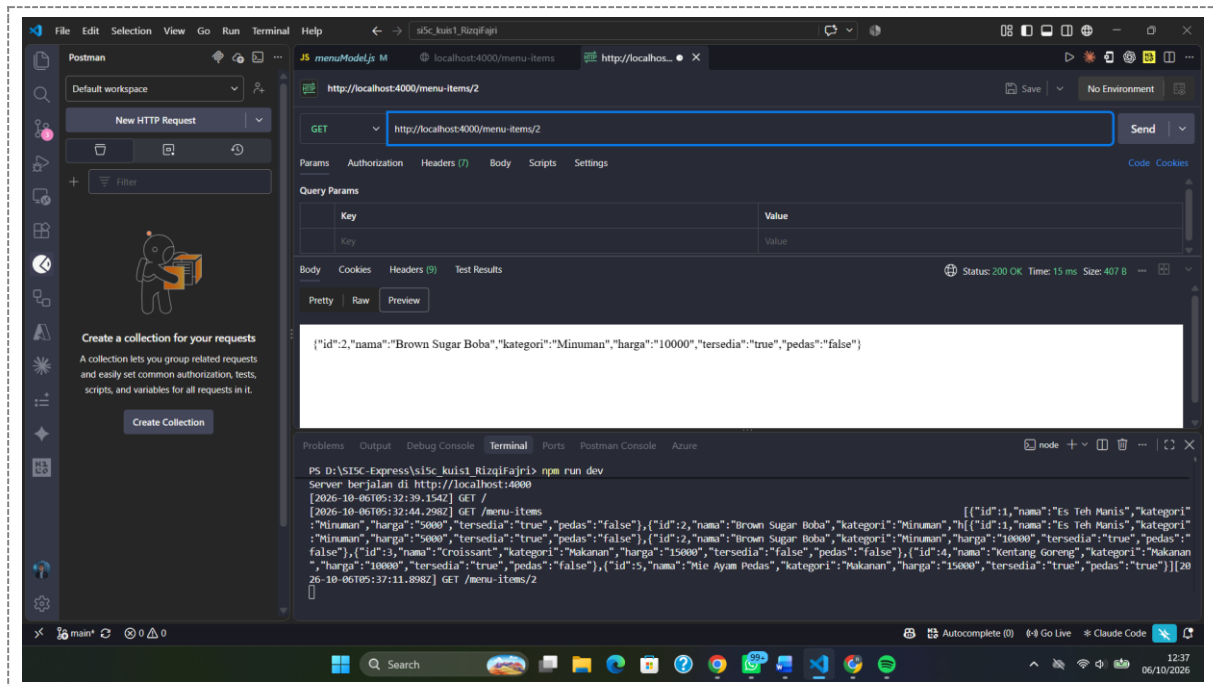
GET `http://localhost:4000/menu-items` berhasil dengan status **200 OK**. Endpoint ini menampilkan daftar menu; tanpa parameter, respons berisi semua 5 menu.

Urutan pemanggilan: route GET / di `menuRoutes.js` → `menuController.getAll` di `menuController.js` → `menuModel.getAll` di `menuModel.js`. Model mengambil data dari daftar menu dalam memori. Endpoint juga bisa memfilter berdasarkan kategori, misalnya `?kategori=Makanan`.

T02 Menampilkan satu data berdasarkan ID

Harus terlihat pada screenshot: ID pada alamat dan objek tunggal pada respons.

Permintaan (request)	GET /menu-items/2
Header	(tidak ada)
Body	(tidak ada)
Hasil yang diharapkan	200 OK; satu objek JSON dengan id yang diminta.
Status hasil uji	<code>{"id":2,"nama":"Brown Sugar Boba","kategori":"Minuman","harga":10000,"tersedia":true,"pedas":false}</code>
Lulus?	Ya



Gambar 5. Hasil GET berdasarkan ID

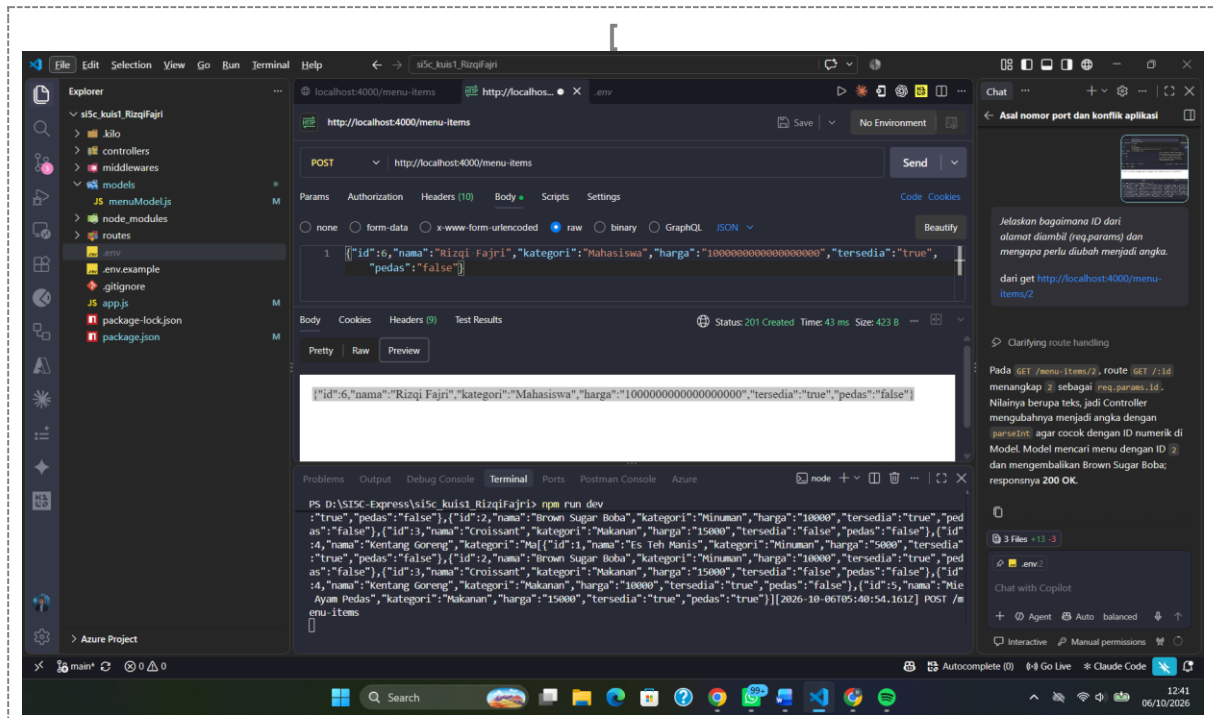
Penjelasan:

Pada GET /menu-items/2, route GET /:id menangkap 2 sebagai req.params.id. Nilainya berupa teks, jadi Controller mengubahnya menjadi angka dengan parseInt agar cocok dengan ID numerik di Model. Model mencari menu dengan ID 2 dan mengembalikan Brown Sugar Boba; responsnya **200 OK**.

T03 Menambah data baru (POST)

Harus terlihat pada screenshot: header x-api-key dan Content-Type, body JSON, status 201, serta objek baru lengkap dengan id. Gunakan data identitas Anda sendiri pada salah satu field teks (misalnya nama atau NIM).

Permintaan (request)	POST /menu-items
Header	x-api-key: kunci-kuis1-paw2 Content-Type: application/json
Body	{\"id\":\"6\",\"nama\":\"Rizqi Fajri\",\"kategori\":\"Mahasiswa\",\"harga\":\"10000000000000000000\",\"tersedia\":\"true\",\"pedas\":\"false\"}
Hasil yang diharapkan	201 Created; objek baru beserta id yang dibuat server.
Status hasil uji	{\"id\":\"6\",\"nama\":\"Rizqi Fajri\",\"kategori\":\"Mahasiswa\",\"harga\":\"10000000000000000000\",\"tersedia\":\"true\",\"pedas\":\"false\"}
Lulus?	Ya



Gambar 6. Hasil POST data baru

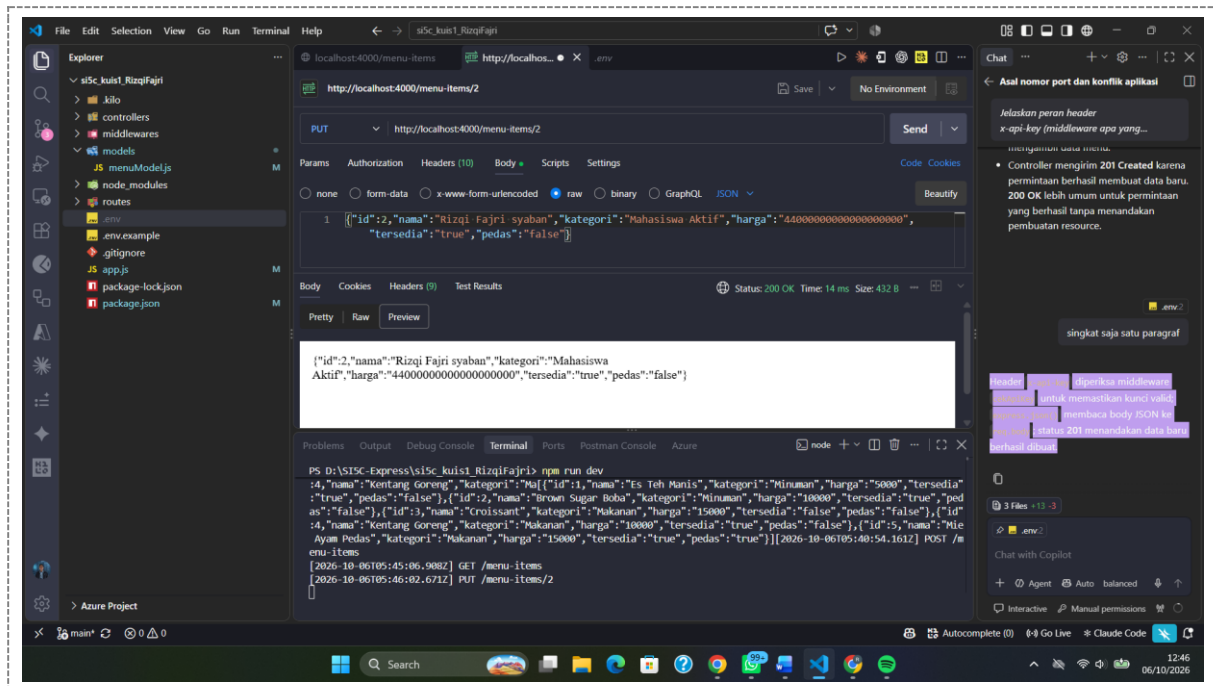
Penjelasan:

Jelaskan peran header `x-api-key` (middleware apa yang memeriksanya), peran `express.json()`, dan alasan status yang dipakai adalah 201, bukan 200.

T04 Mengubah data (PUT)

Harus terlihat pada screenshot: ID data baru pada alamat, header lengkap, body berisi field yang diubah, dan objek hasil perubahan.

Permintaan (request)	PUT /menu-items/2
Header	x-api-key: kunci-kuis1-paw2 Content-Type: application/json
Body	<code>{ "id": 2, "nama": "Rizqi Fajri syaban", "kategori": "Mahasiswa Aktif", "harga": "4400000000000000000", "tersedia": "true", "pedas": "false" }</code>
Hasil yang diharapkan	200 OK; objek dengan field yang sudah berubah.
Status hasil uji	<code>{ "id": 2, "nama": "Rizqi Fajri syaban", "kategori": "Mahasiswa Aktif", "harga": "4400000000000000000", "tersedia": "true", "pedas": "false" }</code>
Lulus?	Ya



Gambar 7. Hasil PUT mengubah data

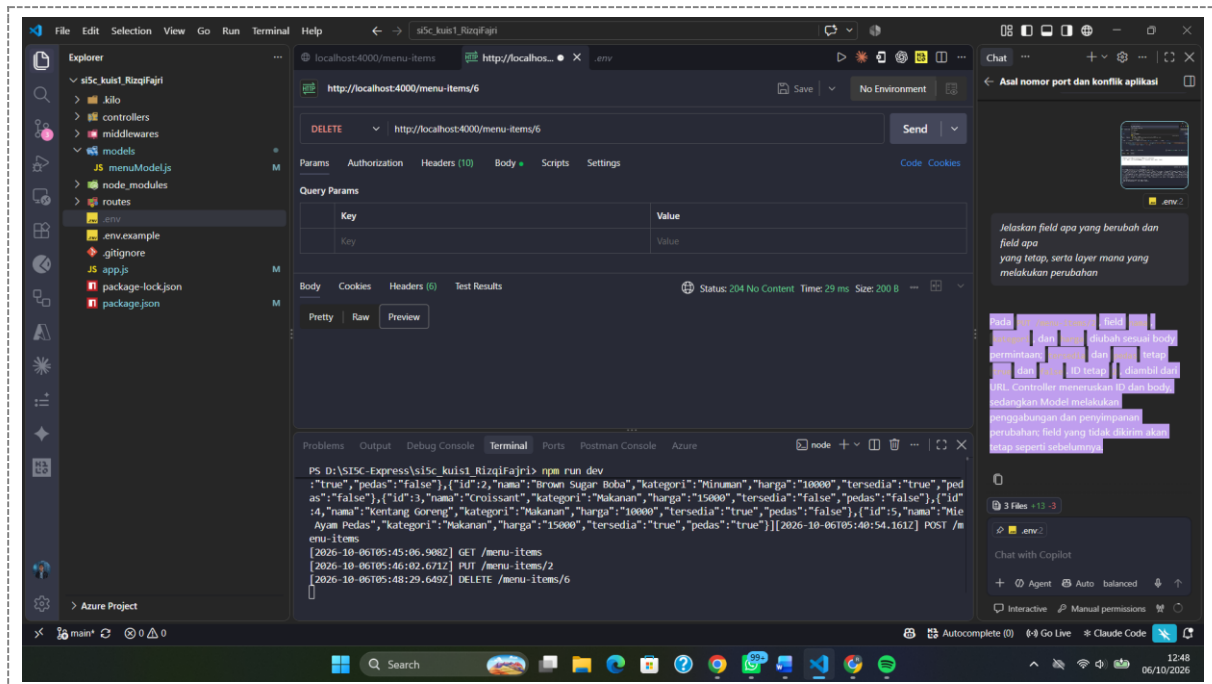
Penjelasan:

Pada PUT `/menu-items/2`, field nama, kategori, dan harga diubah sesuai body permintaan; tersedia dan pedas tetap true dan false. ID tetap 2, diambil dari URL. Controller meneruskan ID dan body, sedangkan Model melakukan penggabungan dan penyimpanan perubahan; field yang tidak dikirim akan tetap seperti sebelumnya..

T05 Menghapus data (DELETE)

Harus terlihat pada screenshot: metode DELETE, header x-api-key, status 204, dan respons tanpa isi.

Permintaan (request)	DELETE <code>/menu-items/6</code>
Header	x-api-key: kunci-kuis1-paw2
Body	(tidak ada)
Hasil yang diharapkan	204 No Content; tanpa isi respons.
Status hasil uji	[2026-10-06T05:48:29.649Z] DELETE <code>/menu-items/6</code>
Lulus?	Ya



Gambar 8. Hasil DELETE data

Penjelasan:

Status **204 No Content** berarti penghapusan berhasil. Respons kosong karena endpoint DELETE hanya mengonfirmasi keberhasilan (`res.status(204).send()`), tanpa mengirim data kembali.

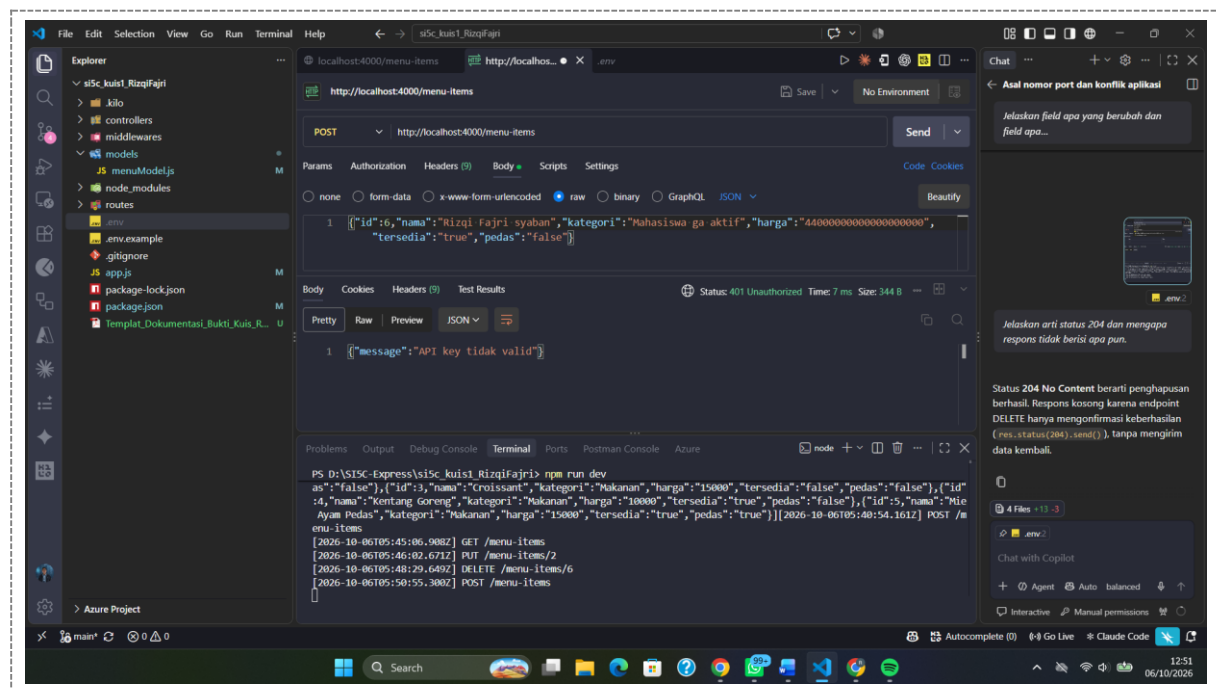
Bagian B. Skenario gagal

Skenario gagal membuktikan bahwa API menolak permintaan yang salah dengan kode status dan pesan yang tepat, bukan runtuh atau membalas halaman HTML berisi jejak error.

N01 Menambah data tanpa API key

Harus terlihat pada screenshot: permintaan POST tanpa header x-api-key dan respons 401.

Permintaan (request)	POST /menu-items (tanpa x-api-key)
Header	Content-Type: application/json
Body	{"message":"API key tidak valid"}
Hasil yang diharapkan	401 Unauthorized; pesan JSON; data tidak bertambah.
Status hasil uji	{"message":"API key tidak valid"}
Lulus?	Ya



Gambar 9. POST tanpa API key ditolak

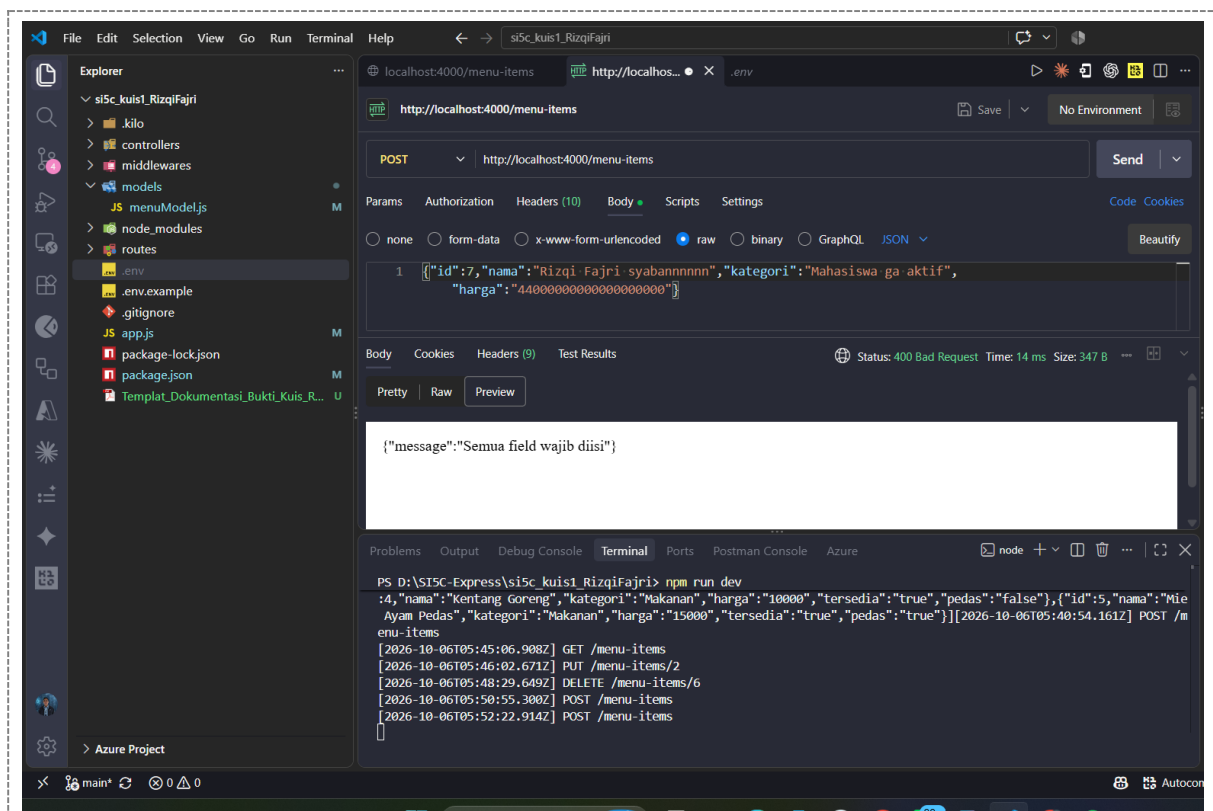
Penjelasan:

Middleware `cekApikey` menolak permintaan karena header `x-api-key` tidak cocok dengan `API_KEY`, lalu mengirim respons **401 Unauthorized**. Middleware tidak memanggil `next()`, jadi Controller tidak dijalankan.

N02 Menambah data dengan body tidak lengkap

Harus terlihat pada screenshot: body yang sengaja kekurangan field wajib dan respons 400.

Permintaan (request)	POST /[sumber-daya]
Header	x-api-key: kunci-kuis1-paw2 Content-Type: application/json
Body	{"message":"Semua field wajib diisi"}
Hasil yang diharapkan	400 Bad Request; pesan menyebut field yang wajib.
Status hasil uji	{"message":"Semua field wajib diisi"}
Lulus?	Ya



Gambar 10. POST dengan body tidak lengkap

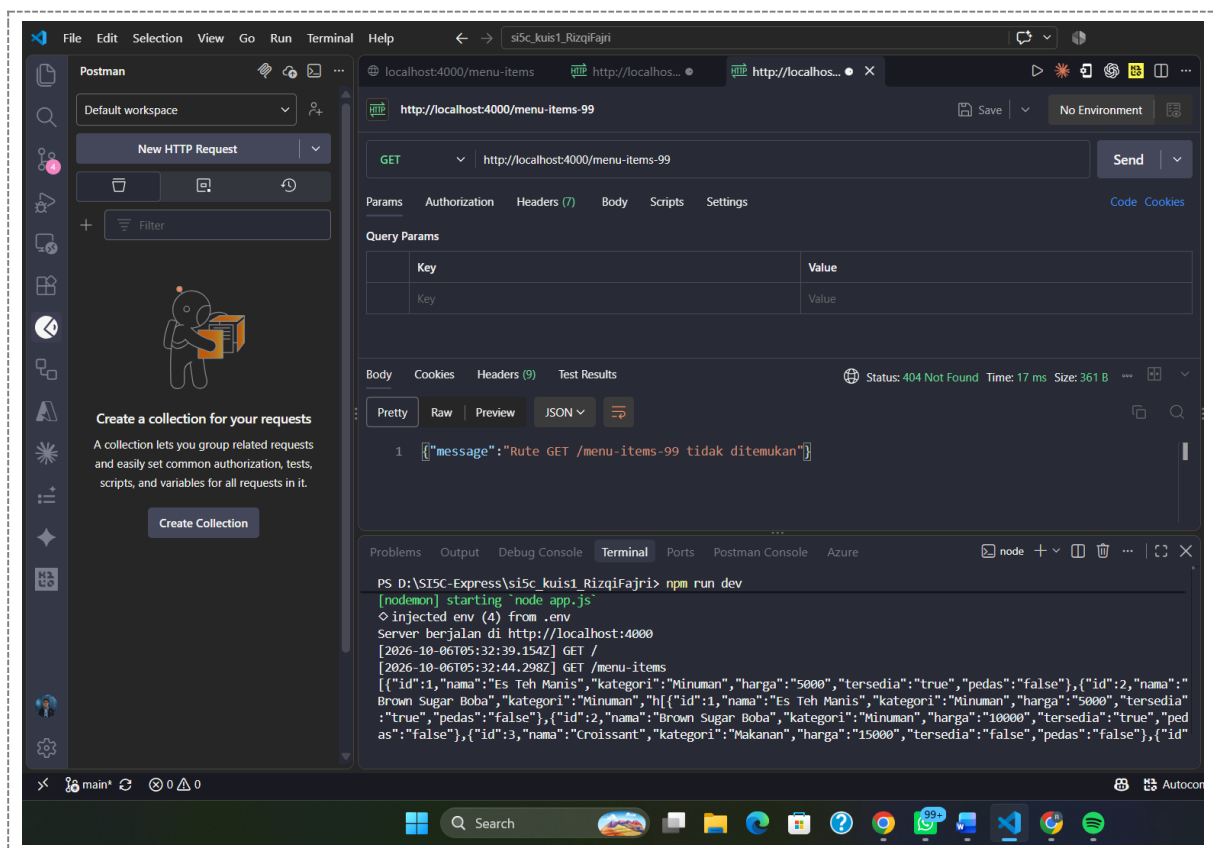
Penjelasan:

Validasi semua field ditulis di **Controller**, pada fungsi `create`. Jika ada field wajib yang kosong, Controller mengirim **400 Bad Request** karena data permintaan tidak lengkap atau tidak valid.

N04 Mengambil ID yang tidak ada

Harus terlihat pada screenshot: ID yang jelas tidak ada (misalnya 99) dan respons 404.

Permintaan (request)	GET /menu-items/99
Header	(tidak ada)
Body	(tidak ada)
Hasil yang diharapkan	404 Not Found; pesan JSON.
Status hasil uji	{"message":"Rute GET /menu-items-99 tidak ditemukan"}
Lulus?	Ya



Gambar 12. GET ID tidak ada

Penjelasan:

Untuk GET /menu-items-99, tidak ada route yang cocok. Setelah route-route dilewati, middleware `notFoundHandler` membuat dan mengirim respons **404**: Rute GET /menu-items-99 tidak ditemukan. Controller tidak dijalankan.

Bab 5 Rekapitulasi Hasil Uji

Salin hasil setiap skenario ke tabel berikut. Tabel ini dibandingkan dosen dengan tangkapan layar pada Bab 4.

ID	Skenario	Status harapan	Status hasil	Lulus
T01	Menampilkan semua data	200	200	Ya
T02	Menampilkan satu data berdasarkan ID	200	200	Ya
T03	Menambah data baru	201	201	Ya
T04	Mengubah data	200	200	Ya
T05	Menghapus data	204	204	Ya
N01	Menambah data tanpa API key	401	401	Ya
N02	Menambah data dengan body tidak lengkap	400	400	Ya
N03	Mengirim JSON yang rusak	400	400	Ya
N04	Mengambil ID yang tidak ada	404	400	Ya

Bab 6 Repositori Kode

Kode dinilai dari repositori GitHub, bukan dari tangkapan layar. Pastikan repositori dapat dibuka dosen (publik atau dosen diundang).

Tautan repositori	https://github.com/rizqifajri25/si5c_kuis1_RizqiFajriSya-ban.git
Pesan commit terakhir	Penambahan file laporan
Folder routes, controllers, models, middlewares ada	Ya
.env tidak terunggah (ada di .gitignore)	Ya
.env.example ada	Ya

Bab 7 Refleksi

Jawablah dengan kalimat Anda sendiri, dua sampai tiga kalimat, dan kaitkan dengan kode topik Anda.

Pertanyaan 1

Jelaskan perbedaan peran Route, Controller, dan Model pada kode Anda. Sebutkan satu contoh baris untuk masing-masing.

Penjelasan:

Route mengarahkan permintaan HTTP ke fungsi yang sesuai, misalnya `router.get('/', menuController.getAll)` di `menuRoutes.js`. Controller menangani permintaan dan respons, contohnya `exports.getAll` di `menuController.js`. Model mengelola data, misalnya `getAll` di `menuModel.js` yang mengambil daftar menu.

Pertanyaan 2

Apa keuntungan error handler terpusat dibandingkan menulis `res.status(404)` di setiap route? Tunjukkan buktinya dari pengujian.

Penjelasan:

*Error handler terpusat membuat format respons kesalahan konsisten dan menghindari penulisan penanganan 404 berulang di tiap route. Buktinya, saat diuji dengan `GET /menu-items-99`, aplikasi mengembalikan **404 Not Found** beserta pesan JSON. Rute `GET /menu-items-99` tidak ditemukan dari `notFoundHandler`.*

Lampiran A Daftar Periksa Sebelum Mengumpulkan

No	Butir yang diperiksa	Ya
1	Seluruh tanda [. . .] sudah diganti dengan topik Tugas 1 saya	Ya
2	Semua tangkapan layar berasal dari komputer saya dan terbaca jelas	Ya
3	Setiap gambar memiliki nomor, keterangan, dan penjelasan	Ya
4	Identitas saya (nama atau NIM) muncul pada data uji POST	Ya
5	Sembilan skenario pada Bab 4 sudah lengkap, termasuk status hasil uji	Ya
6	Tabel rekapitulasi sesuai dengan tangkapan layar	Ya
7	Tautan repositori pada sampul dapat dibuka dan .env tidak terunggah	Ya
8	Berkas disimpan sebagai PDF dengan nama sesuai ketentuan	Ya

Lampiran B Rubrik Penilaian (Usulan)

Bobot di bawah ini adalah usulan awal dan dapat diubah dosen sebelum dibagikan ke mahasiswa.

Komponen	Kriteria penilaian penuh	Bobot
Instalasi dan struktur folder (Bab 1 sampai 3)	Instalasi, struktur folder, dan server berjalan benar	15
Ringkasan kode dan kode di repositori (Bab 2 dan 6)	Tanggung jawab tiap layer tidak tercampur; penjelasan menyebut kode sendiri; .env tidak terunggal	30
Pengujian skenario berhasil (T01 sampai T05)	Status dan respons sesuai; penjelasan benar	25
Pengujian skenario gagal (N01 sampai N04)	Kode status dan pesan JSON sesuai; penjelasan alur error benar	20
Rekapitulasi dan refleksi (Bab 5 dan 7)	Rekap konsisten dengan bukti; jawaban pribadi	10
Total		100